Relatório técnico — BootCamp III | Fase 2

Identificação

- Projeto: **bootcamp_task_analyzer**
- Repositório: <https://github.com/Bruno-Kowalski/bootcamp_task_analyzer>
- Branch avaliada: **main**
- Data da atualização: 17/10/2026
- Pull Requests históricos:

- [PR #1 — implementação](https://github.com/Bruno-Kowalski/bootcamp_task_analyzer/pull/1)
- [PR #2 — instruções de execução](https://github.com/Bruno-Kowalski/bootcamp_task_analyzer/pull/2)

- Status dos Pull Requests: fechados/mesclados

Os PRs históricos documentam a evolução inicial do projeto. O PR #2 registra **15 testes aprovados**, que correspondia ao estado daquele momento. Após a revisão do contrato da Fase 2, a suíte atual foi simplificada para os três grupos de aceite solicitados e possui cinco casos executados com sucesso.

1. Objetivo e fonte de verdade

O TaskAnalyzer é um módulo Python em memória, orientado pela especificação SDD em [**specs/task_analyzer_spec.md**](../specs/task_analyzer_spec.md) e pelas regras de governança em [**CONTEXT_RULES.md**](../CONTEXT_RULES.md).

O contrato atual define:

- totais gerais de tarefas, concluídas e pendentes;
- média do tempo de conclusão em horas;
- taxa de atraso percentual;
- indicadores gerais e agrupados por prioridade;
- **TaskValidationError** para datas inconsistentes;
- retorno de médias **0.0** para entradas vazias ou sem tarefas concluídas.

2. Implementação

O módulo [**src/task_analyzer.py**](../src/task_analyzer.py) fornece:

- **Task**, **AnalysisResult** e **PriorityIndicator** como dataclasses;
- **Prioridade** e **Status** como enums;
- **analyze_tasks(tasks: list[Task]) -> AnalysisResult**;
- **TaskValidationError** com a mensagem **Datas inválidas ou inconsistentes**;
- validação de **data_conclusao < data_criacao**;
- validação de datas timezone-aware em UTC;
- validação de identificadores positivos e únicos;
- cálculo de médias em horas, atrasos e indicadores por prioridade;
- logging por meio da biblioteca padrão.

A implementação utiliza Python 3.11+ compatível, type hints, docstrings, funções coesas e tratamento específico de erros.

3. Test Harness e homologação

O arquivo [**tests/test_harness.py**](../tests/test_harness.py) utiliza **pytest** e traduz diretamente os cenários exigidos:

1. **Sucesso:** valida **total_tarefas**, **total_concluidas**, **total_pendentes**, média geral em horas, média por prioridade e taxa de atraso;
2. **Exceção/erro:** valida que **data_conclusao < data_criacao** dispara **TaskValidationError**;
3. **Casos de borda:** valida entrada vazia e entrada somente com tarefas pendentes, ambas retornando médias e taxa de atraso iguais a **0.0**.

Comando executado:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Resultado atual:

```
5 passed in 0.02s
```

Também foi executado:

```
.\.venv\Scripts\python.exe -m compileall -q .\src .\tests
```

Resultado: compilação concluída sem erros.

4. Rastreabilidade

Exigência	Implementação/teste
Cenário 1	**test_analise_calcula_totais_medias_e_atrasos**
Cenário 2	**test_data_inconsistente_gera_erro**
Entrada vazia	**test_entrada_sem_concluidas_retorna_zeros[entrada_vazia]**
Somente pendentes	**test_entrada_sem_concluidas_retorna_zeros[somente_tarefas_pendentes]**
Média por prioridade sem concluídas	**test_media_da_prioridade_sem_concluidas_e_zero**
Contrato funcional	**specs/task_analyzer_spec.md**
Regras de governança	**CONTEXT_RULES.md**

5. Uso de IA e regras de contexto

O desenvolvimento assistido por IA foi conduzido com consulta prévia à especificação e ao **CONTEXT_RULES.md**. As alterações foram revisadas para manter:

- Python 3.11+ e type hints;
- aderência ao contrato funcional;
- tratamento específico de **TaskValidationError**;
- testes derivados dos cenários de aceite;
- ausência de persistência de dados;
- documentação e evidências atualizadas.

A homologação final deve considerar o estado atual da **main**, e não somente os

resultados históricos registrados nos PRs anteriores.

6. Versionamento e Pull Requests

Os PRs #1 e #2 foram mesclados na **main** e demonstram o fluxo de versionamento Git/GitHub. O histórico da **main** também contém commits de atualização do contrato, da implementação, dos testes e da documentação.

O README atual e este relatório identificam explicitamente a diferença entre a evidência histórica de **15 passed** e a validação atual de **5 passed**.

7. Evidências

As evidências versionadas incluem:

- [**docs/evidence_pytest.txt**](./evidence_pytest.txt), com a execução atual;
- [**docs/evidence_pytest.png**](./evidence_pytest.png), captura visual da execução dos testes;
- [**docs/evidence_tree.txt**](./evidence_tree.txt), árvore do repositório;
- [**docs/evidence_tree.png**](./evidence_tree.png), captura visual da árvore;
- [**docs/evidence_pull_request.png**](./evidence_pull_request.png), evidência visual do fluxo de Pull Request;
- este relatório técnico.

7.1 Prints comprobatórios

Os prints abaixo foram produzidos a partir dos artefatos versionados do projeto:

Execução do Test Harness

```
============================= test session starts =============================
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\bruno\bootcamp_task_analyzer\.venv\Scripts\py
cachedir: .pytest_cache
rootdir: C:\Users\bruno\bootcamp_task_analyzer
collecting ... collected 5 items

tests/test_harness.py::test_analise_calcula_totais_medias_e_atrasos PASSED [ 20%]
tests/test_harness.py::test_data_inconsistente_gera_erro PASSED          [ 40%]
tests/test_harness.py::test_entrada_sem_concluidas_retorna_zeros[entrada_vazia] PASSED [ 60%]
tests/test_harness.py::test_entrada_sem_concluidas_retorna_zeros[somente_tarefas_pendentes] PASSED [ 80%]
tests/test_harness.py::test_media_da_prioridade_sem_concluidas_e_zero PASSED [100%]

============================== 5 passed in 0.02s ==============================
```

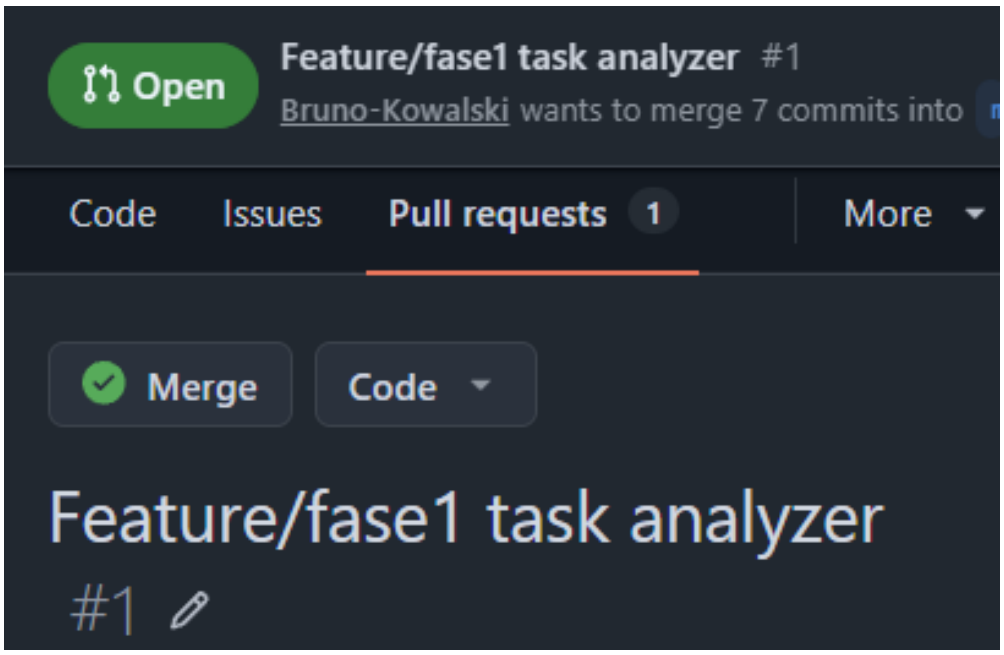
A imagem comprova a execução dos cinco casos atuais, todos com status **PASSED**, e o resumo final **5 passed**.

Estrutura do repositório



A imagem comprova a presença de **specs/**, **tests/**, **src/**, **README.md**, **CONTEXT_RULES.md** e **docs/**.

Fluxo de Pull Request



A imagem comprova o uso do fluxo de Pull Request no GitHub. O conteúdo textual do PR é histórico e deve ser interpretado junto com a validação atual publicada na **main**.

8. Conclusão

Na validação atual, a estrutura exigida está presente, a implementação segue o contrato SDD revisado e o Test Harness cobre os cenários de sucesso, erro e borda. A suíte executa com aprovação total:

```
5 passed
```

O PDF técnico é um artefato binário histórico; este arquivo Markdown é a fonte editável e atualizada do relatório.

9. Revisão técnica de conformidade

Foi realizada uma auto-revisão técnica do estado publicado na **main** em 17/10/2026. A revisão comparou o barema com a árvore do repositório, a especificação, a implementação, o Test Harness e as evidências versionadas.

Item revisado	Verificação	Resultado
Estrutura exigida	**specs/**, **tests/**, **src/**, **README.md** e **CONTEXT_RULES.md** presentes	Aprovado
Versionamento	**main** publicada, histórico de commits preservado e PRs históricos documentados	Aprovado
Implementação SDD/IA	**src/task_analyzer.py** segue a especificação, usa type hints e tratamento específico de erros	Aprovado
Cenário 1	Totais, médias gerais, médias por prioridade e taxa de atraso validados	Aprovado
Cenário 2	**TaskValidationError** validado para data de conclusão anterior à criação	Aprovado
Casos de borda	Entrada vazia e somente pendentes retornam médias **0.0** sem erro não tratado	Aprovado
Evidência de execução	**docs/evidence_pytest.txt** e **docs/evidence_pytest.png** registram **5 passed**	Aprovado
Consistência documental	README, especificação, regras, relatório e evidências usam o contrato atual	Aprovado

Resultado da auto-revisão

Conforme para entrega segundo os critérios técnicos avaliados.

Esta é uma revisão técnica realizada pelo autor do projeto e está sendo documentada para atender ao requisito de revisão/documentação do trabalho. Ela não deve ser apresentada como aprovação independente de um segundo usuário do GitHub. Os PRs históricos permanecem preservados como parte do versionamento e suas descrições antigas são identificadas neste relatório como evidências históricas, não como resultado da validação atual.

10. Relato estruturado de homologação assistida por IA

10.1 Objetivo

Homologar se a implementação, os testes e a documentação atendem ao contrato SDD e aos critérios de avaliação da Fase 2, sem substituir a responsabilidade do autor pela revisão final.

10.2 Insumos utilizados

- [**specs/task_analyzer_spec.md**](../specs/task_analyzer_spec.md);
- [**CONTEXT_RULES.md**](../CONTEXT_RULES.md);
- [**src/task_analyzer.py**](../src/task_analyzer.py);
- [**tests/test_harness.py**](../tests/test_harness.py);
- [**README.md**](../README.md);
- evidências textuais e visuais em **docs/**;
- histórico Git/GitHub da branch **main**.

10.3 Procedimento de homologação

1. Comparação do contrato implementado com a especificação SDD.
2. Verificação de type hints, dataclasses, enums e tratamento de erros.
3. Conferência da correspondência entre os cenários de aceite e os testes.
4. Execução do Test Harness com **pytest -v**.
5. Execução de compilação com **compileall**.
6. Conferência de consistência documental e das evidências visuais.

7. Verificação do estado publicado na branch **main**.

10.4 Evidências de execução

Verificação	Comando/artefato	Resultado
Test Harness	**python -m pytest -v**	**5 passed**
Compilação	**python -m compileall -q src tests**	Aprovada
Integridade do diff	**git diff --check**	Aprovada
PDF atualizado	**docs/relatorio_tecnico_fase2.pdf**	Válido e visualmente conferido
Repositório publicado	**origin/main**	Sincronizado

10.5 Resultado e limitações

Homologado para entrega acadêmica nos critérios técnicos avaliados.

Foram confirmados os cenários de sucesso, exceção e borda, além da consistência entre código, especificação, testes, README, regras de contexto e relatórios. A homologação foi realizada com assistência de IA e revisão final do autor.

Esta homologação não representa uma aprovação independente de outro usuário do GitHub. O requisito de revisão/documentação foi atendido por meio do histórico de Pull Requests, da auto-revisão técnica registrada e dos prints incluídos neste relatório.